
Let's Make Pottery: Recover Beautiful Pottery From Broken Chips Based on GANs

Yuchao Jin
2300012908

Jishan Cai
2300012926

Abstract

Accurate completion of archaeological artifacts is a critical aspect in several archaeological studies, including documentation of variations in style, inference of chronological and ethnic groups, and trading routes trends, among many others. However, most available pottery is fragmented, leading to missing textural and morphological cues. To address these issues and support archaeologists in their work, we developed a 3D Autoencoder Generative Adversarial Network (3D AE-GAN) with the provided framework. The model was trained on a dataset containing complete restoration references for pottery fragments. To evaluate the effectiveness of the proposed approach, we conducted comprehensive quantitative analyses. Additionally, a random selection of generated outputs was visualized, demonstrating the model's ability to accurately reconstruct missing regions and preserve rough details.

1 Introduction

Archaeological excavations frequently yield damaged artifacts, such as pottery fragments, whose restoration is both critical and challenging. The restoration process not only aids in recovering the artifact's original appearance but also provides invaluable insights into the historical, cultural, and economic contexts of the artifact's origin. Traditional restoration methods are labor-intensive, requiring skilled expertise and substantial time investment, which often slows down archaeological research and limits the scope of study.

Furthermore, manual techniques can be prone to inconsistencies and subjective interpretations, introducing variability into the results. With advancements in machine learning and computer vision, automated approaches for artifact restoration have emerged as a promising alternative[1]. Such methods have the potential to accelerate the restoration process, enhance consistency, and facilitate the analysis of large collections of fragmented artifacts. Nevertheless, the development of effective computational models for artifact restoration poses its own challenges. These include handling irregular and incomplete input data, preserving fine morphological and textural details, and ensuring that reconstructions remain consistent with archaeological contexts. Addressing these challenges is crucial for creating reliable and practical restoration tools to support archaeologists in their work.

To this end, we implement a 3D Autoencoder Generative Adversarial Network (3D AEGAN) based on the given code framework, specifically designed to restore fragmented pottery artifacts. By leveraging a dataset of complete restoration references, the model is trained to learn the intricate patterns and structures characteristic of archaeological pottery, enabling it to reconstruct missing regions reasonably.



2 Related Work

Artifact restoration has been a subject of extensive research in archaeology and computer science. Traditional manual approaches, while reliable, are highly dependent on expert knowledge and often involve painstaking physical reconstruction. The advent of computational techniques has introduced a new paradigm for restoration tasks, with approaches ranging from image-based reconstruction to 3D modeling.

In the realm of 2D restoration, image inpainting techniques have demonstrated significant progress. Notable methods, such as Contextual Attention GANs[2] and Partial Convolutions[3] have shown promise in reconstructing missing regions in 2D images. However, these methods fall short when applied to 3D objects, as they fail to capture the volumetric and structural intricacies inherent in 3D data.

For 3D restoration, traditional methods often rely on geometric heuristics, such as point cloud alignment[4] or mesh reconstruction[5], which are limited in handling irregular or severely damaged inputs. Recent advancements in deep learning have spurred interest in neural network-based approaches for 3D data processing. Autoencoders and GANs have been successfully applied to tasks like shape completion and surface reconstruction. For example, 3D-EPN[6] and AtlasNet[7] have demonstrated the potential of learning-based methods in handling 3D point clouds and meshes.

Despite these advancements, challenges remain in integrating structural and textural information for seamless artifact restoration. Existing methods often overlook fine details or fail to generalize across diverse artifact types. Our proposed framework also use GANs with Autoencoders and change the order of Discriminator and Generator enabling reasonable restoration of fragmented pottery artifacts.

3 Data

The datasets we use consist of .vox files generated from real pottery models without special processing, comprising 11 different types. These files adhere to the MagicaVoxel format, where each file may contain fragments labeled with positive integers, with 0 indicating an empty space. The amount of training data is approximately 11,000 entries. We can take a look at these data through our methods (Figure 1, Figure 2).

4 Proposed Approach

We employ a Generative Adversarial Network (GAN) to reconstruct complete pottery from fragments and to classify their authenticity. The generator G is responsible for reconstructing the original pottery from fragments, while the discriminator D determines whether a piece of pottery is real or generated by the model. (Figure 3)

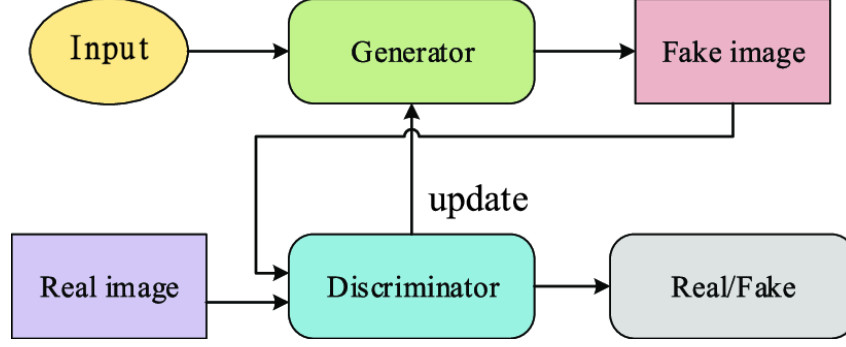


Figure 3: Model

4.1 Generator

The generator comprises an encoder-decoder structure.

The encoder takes input of dimensions $32 \times 32 \times 32$ or $64 \times 64 \times 64$ and outputs a 128-dimensional vector, representing the features of the original pottery. The encoder consists of five convolutional layers. The first four layers include convolution operations with kernel size = 4, stride = 2, padding = 1, and doubling the number of channels, followed by batch normalization and a LeakyReLU activation function. The fifth layer is a convolutional layer without batch normalization or activation, and its padding depends on the input size: padding = 1 for selected resolution equals to 32 and padding = 0 for resolution equals to 64.

The decoder takes a 128-dimensional vector as input and generates a $32 \times 32 \times 32$ or $64 \times 64 \times 64$ voxel representation of the pottery. The decoder is also composed of five layers. The last four layers include transposed convolutions with kernel size = 4, stride = 2, padding = 1, and doubling the number of channels, followed by batch normalization and ReLU activation. The first layer is a transposed convolutional layer without batch normalization or activation, with padding depending on the input size: padding = 1 for selected resolution equals to 32 and padding = 0 for resolution equals to 64.

This architecture creates a mirrored correspondence between the encoder and decoder.

4.2 Discriminator

The discriminator takes $32 \times 32 \times 32$ or $64 \times 64 \times 64$ input and outputs a value between 0 and 1, representing the confidence that the input is authentic pottery. Its structure is similar to the encoder, but with the following two differences: the LeakyReLU layers use a larger slope parameter, and a sigmoid layer is added at the end to map the 128-dimensional feature vector to a scalar value between 0 and 1, representing the confidence score.

5 Experiments

5.1 Training Strategy

In principle, GANs train D and G simultaneously, using BCE loss function to guide D in distinguishing real pottery from generated ones and to guide G in producing pottery that can fool D . In our practice, we set the resolution of training data $64 \times 64 \times 64$.

Unlike traditional GANs (Figure 4), which often train D for multiple iterations before training G , we adopt an alternating approach, training G first and then D . This strategy stems from the observation that GANs training is fundamentally a dynamic game where the generator and discriminator must co-evolve. Training G first prevents D from converging too quickly, which could make it challenging for G to learn effective generation strategies. By alternating training, we maintain a relative balance in the learning pace of G and D , reducing instability during training and enhancing coherence.

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Figure 4: Traditional GANs training strategy

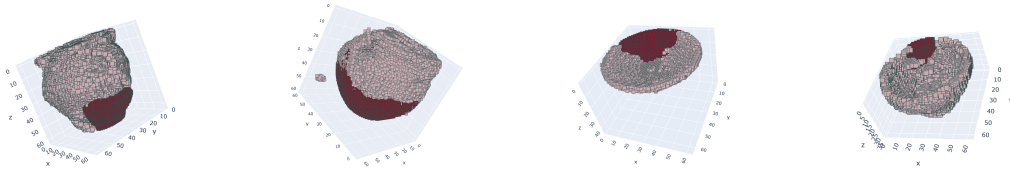


Figure 5: Generate Pottery From Our Model

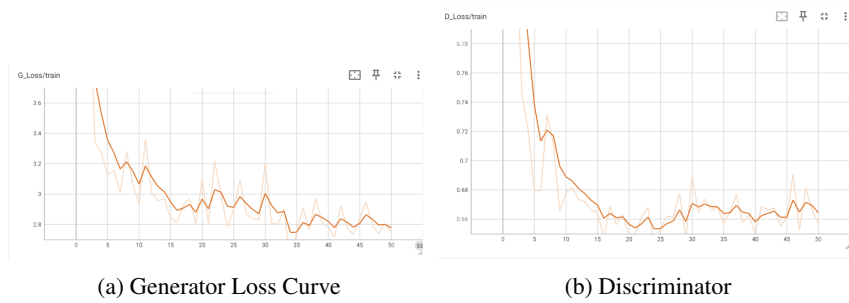


Figure 6: Loss Curves

Table 1: Performance metrics across different ceramic types of pottery generated by our model

class index	DCS>0.1	JD<0.9	MSE
class 1	75%	12.5%	0.07399
class 2	100%	92.30%	0.1300
class 3	90%	0%	0.06429
class 4	100%	0%	0.08753
class 5	100%	100%	0.1212
class 6	100%	33.33%	0.08055
class 7	78.57%	57.14%	0.10967
class 8	100%	64.20%	0.09561
class 9	100%	33.33%	0.1242
class 10	29.35%	2.174%	0.04338
class 11	3.448%	0%	0.0340

5.2 Training result

Here are our Training loss curve(Figure 6). And we choose some of pottery generated by our model(Figure 5). We also tested our model with different evaluation functions(Table 1).

5.3 Try improvement methods

We attempted several improvement methods.

1. Transforming the Generator’s Output into a Binary Matrix. Intuitively, the output was a binary matrix, which aligns better with the form of frag + G(frag). However, we found that the output of G contained unreasonable results such as negative values and values greater than 1.0 To address this, we applied a sigmoid activation before G ’s output to constrain it. The result was that the generated pots were extremely sparse, containing only a rough outline. After experimenting with various sigmoid modifications (e.g., shifting by -0.5 or $x-0.5$), we concluded that the results are inferior to the original model’s output.
2. Modifying the Loss Function. We attempted to add voxel loss (directly comparing the voxel differences between the generated model and the original pottery) to the G ’s loss function. Additionally, we experimented with combining DCSloss, JDloss, and MSEloss as the discriminator’s loss function. The training results showed that the generator seemed to recognize every voxel as part of the pottery, resulting in nearly solid-filled pottery that occupied the entire space.
3. Adding Labels and Classification Training. This was motivated by heavy classification errors, because we noticed that a bowl fragment was completed as a pot. We considered adding the label to the encoded feature vector to participate in the decoding process, but this may broke the network structure, such as the in and out channels of convolution. We also tried to train only in a small dataset which only contains one label. However, this led to severe overfitting due to the size of selected dataset. In the end, we choose to mix the datasets with 11 labels for training.

6 Conclusion

The restoration of pottery plays a vital and frequent role in archaeological work, making our project highly meaningful in practice. Beyond its archaeological applications, AEGAN (Autoencoder GAN) holds potential in fields like biochemistry, where it could predict the structure of biomolecules such as proteins—a promising direction for future exploration.

Throughout the development of this project, we gained a deeper understanding of the GAN algorithm covered in class. Building on the convolutional concepts taught, we extended our knowledge to include transposed convolutions. We also adapted the ResNet block-based design principles from Assignment 6 to the modular design of GANs. Additionally, we learned techniques such as regularization matching and advanced file handling, utilized the Plotly library for data and result visualization, and encountered challenges like PyTorch’s computational graph being cleared after a backward pass in a batch, causing errors upon reuse—an issue we addressed for the first time.

Furthermore, we explored remote training by connecting to remote hosts and collaborated using Overleaf for multi-user writing.

All in all, we find that the modern neural network is harder than we thought, since it has big time cost and high complexity. GANs is much more difficult than any other models we have tried while brought us much fun in the whole process of implement this final project.

7 Teamwork

Yuchao Jin implement Discriminator and Generator model, implement the train process, implement data loader and visualization.

Jishancai implement test part, try and apply some modification in train process, implement visualizing the result

code base <https://drive.google.com/file/d/1sMqk8f2FuIBX3oo9HFNRGn4KUFWc097F/view?usp=sharing>

References

- [1] Renato Hermoza and Ivan Sipiran. 3d reconstruction of incomplete archaeological objects using a generative adversarial network. In *Proceedings of Computer Graphics International 2018*, CGI 2018, page 5–11, New York, NY, USA, 2018. Association for Computing Machinery.
- [2] Jiahui Yu, Zhe Lin, Jimei Yang, Xiaohui Shen, Xin Lu, and Thomas S. Huang. Generative image inpainting with contextual attention, 2018.
- [3] Guilin Liu, Fitsum A. Reda, Kevin J. Shih, Ting-Chun Wang, Andrew Tao, and Bryan Catanzaro. Image inpainting for irregular holes using partial convolutions, 2018.
- [4] Ziyu Guo, Renrui Zhang, Xiangyang Zhu, Yiwen Tang, Xianzheng Ma, Jiaming Han, Kexin Chen, Peng Gao, Xianzhi Li, Hongsheng Li, and Pheng-Ann Heng. Point-bind & point-llm: Aligning point cloud with multi-modality for 3d understanding, generation, and instruction following, 2023.
- [5] Zhiqin Chen. A review of deep learning-powered mesh reconstruction methods, 2023.
- [6] Angela Dai, Charles Ruizhongtai Qi, and Matthias Nießner. Shape completion using 3d-encoder-predictor cnns and shape synthesis, 2017.
- [7] Thibault Groueix, Matthew Fisher, Vladimir G. Kim, Bryan C. Russell, and Mathieu Aubry. Atlasnet: A papier-mâché approach to learning 3d surface generation, 2018.